Functional Spec

Functional Specification

Purpose

The CDTA 2026 Dashboard is a single-page web application that displays the complete state of the **Coimbatore District Tennis Association 2026 League** — a doubles tennis league with 11 divisions (A–K).

It is the public-facing read view, the captains' planning tool, and the admin's operational console.

Users and roles

Role	Sees	Typical user
admin	All 11 divisions + Overview + SVG Players + Admin tab	League secretary
captain	All 11 divisions + Overview + SVG Players	Team captains
svgplayer	All 11 divisions + Overview + SVG Players	SVG club members
viewer	All 11 divisions + Overview + SVG Players	Anyone with the link

Only `admin` sees the **Admin** tab. All other roles see the same content but with no admin features.

Authentication is local to `data/auth.js` (no server session). On successful login the role and user info are stored in `localStorage`.

Page inventory

Per division (A through K) — 5 sub-tabs each

Sub-tab	Content
Standings	Final ranks, courts played, points, sets W/L, games W/L, win %

| **Projection** | (Manual divisions C/F/H/I only) — projected final position |

| **Schedule** | Week-by-week fixtures with court-level player pairings & set scores |

| **Results** | Match-by-match results, filterable by week and team |

| **Players** | Roster with photo, role, played courts, wins/losses, win %, partners |

| **Stats** | (C/F/H/I) — Most Active Doubles Pairs, top performers |

| **Teams** | Team list with captain & player counts |

Overview (top-level menu)

Consolidated dashboard:

- **Leaders** — top scorers across all divisions
- **All Teams** — every team across all 11 divisions
- **League Stats** — totals row (teams, players, matches, courts)

SVG Players (top-level menu)

Special club view focused on the SVG Tennis Club's three teams (SVG-C in Div C, plus STARS and SAI in Div I). Sub-tabs:

- **Club Performance** (default)
- **Players**
- **Schedule**
- **Results**

Admin (admin role only) — 7 sub-tabs

| Sub-tab | Purpose |

|---|---|

| **User Management** | Create / edit / delete users |

| **Login History** | Who logged in when |

| **Usage Log** | Page views per user |

| **Activity Log** | Mutations (data tool runs, etc.) |

| **Test Plan** | Embedded test checklist |

| **Data Tools** | Auto-fix stats, integrity check, refresh helpers |

| **Settings** | Total weeks, league name, photo paths, etc. |

Key user flows

1. View standings (any user)

1. Login → land on Overview
2. Click **Div A** (or any letter A–K)
3. See **Standings** tab by default
4. Click any column header to sort

2. Inspect a player (any user)

1. Open a division → **Players** tab
2. Filter by team / search by name / filter by gender
3. Row shows: photo, name, team, gender, courts played, W/L, win %, partner pairings

3. Refresh data after a week (admin)

See ADMIN_RUNBOOK.md. High-level:

1. Run `/load-division` skill (per division), or use **Data Tools → Auto Fix Stats**
2. Verify in app
3. Push to GitHub
4. Run `deploy.bat` to publish to GoDaddy

4. Add a new user (admin)

1. **Admin → User Management → + Add User**
2. Enter username, password, role, division (optional), team (optional)
3. Saved to `data/auth.js` via `api/admin-data.php`

Display rules

Final Standings

- Section title: `Division X — Final Standings` (season is over)
- Header stat box: `0 Weeks Remaining`
- Win % color: ≥80% green, ≥50% muted, <50% red

Player Stats (auto-derived on init)

- `gamesPlayed` = court appearances (counted from `DIV_X_SCHEDULE`)
- `wins` = court wins (winning side of each court)

- `partners` = list of `[partner_name, count]` per player, sorted desc
- `KEY_PAIRS.wins` = re-derived per pair
- Recomputation runs at page load — no manual maintenance

Manual vs scraped divisions

- **Scraped** (A, B, D, E, G, J, K) — data pulled from league.cdta.co.in via the `/load-division` skill
- **Manual** (C, F, H, I) — same base structures **plus**: `PROJECTIONS`, `PROJ_BREAKDOWN`, `KEY_PAIRS`, `PARTNERS`, `TEAM_INFO`

Out of scope (intentional non-features)

- No write API for non-admin users
- No live score updates (data is refreshed weekly)
- No mobile app (responsive web only)
- No payment / registration / signup
- No multi-language support

Technical Spec

Technical Specification

Overview

CDTA 2026 Dashboard is a **single-page client-rendered web app** with no build step. It is plain HTML + vanilla JavaScript + CSS, served as static files from GoDaddy. A small PHP backend (`api/admin-data.php`) provides persistence for admin-modified data.

There is no framework, no bundler, and no transpiler. All logic lives in `js/app.js` (~5,200 lines).

Architecture

```
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ Browser ■
■ ■■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■■■■ ■
■ ■ index.html ■ → ■ data/*.js ■ → ■ js/app.js ■ ■
■ ■ (DOM shell) ■ ■ (globals) ■ ■ (renders DOM) ■ ■
■ ■■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■■■■ ■
■ ↑ ■
■ ■ admin writes ■
■ ↓ ■
■ ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■
■ ■ api/admin-data.php (server, GoDaddy) ■ ■
■ ■ - read/write JSON state for admin features ■ ■
■ ■ - login history, usage log, settings, etc. ■ ■
■ ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
```

Load order

1. `index.html` loads all 11 `data/division-{a..k}.js` files plus `data/auth.js` as `<script>` tags. Each file declares globals like `DIV_C_PLAYERS`, `DIV_C_SCHEDULE`, `DIV_C_PARTNERS`.

2. `js/app.js` runs after all data is loaded. It defines render functions and immediately calls them at the bottom of the file.

3. Each division has its own init block (`renderDivXStandings()`, `renderDivXTeams()`, etc.).

Key modules in `js/app.js`

Section	Lines (approx)	What it does
Helpers (`_esc`, `_standingRowClass`, etc.)	top	Shared utilities
Authentication & menu	1–400	Login, role enforcement, menu visibility
Division I (default)	400–800	The original division — `renderStandings()`, `renderPlayers()`, `renderResults()`, etc.
Division C	600–3000	Manual division with projections, key pairs, partners
`recomputeDivStats()`	~3070	**Critical helper** — derives player stats from schedule courts
Divisions F, H	3080–3700	Manual divisions

| Divisions A, B, D, E, G, J, K | 3700–4500 | Scraped divisions (rendering only — data is pre-loaded) |

| Admin panel | 1500–2700 | User mgmt, logs, data tools, integrity check, auto-fix |

| Overview | 900–1300 | Cross-division consolidation |

| SVG Players club view | 2900–3060 | Special club view |

The recompute pipeline (most important algorithm)

After a season refresh, the static `gamesPlayed` and `wins` fields in `DIV_X_PLAYERS` may be stale or `null`. Rather than store these values, **they are derived at runtime** from the authoritative source — the per-court records in `DIV_X_SCHEDULE`.

Function: `recomputeDivStats(SCHEDULE, PLAYERS, PARTNERS, KEY_PAIRS?)`

Located in `js/app.js`, called once per manual division (C, F, H, I) before any render.

Steps:

1. Build a name normalization map: `normalize(name)` → strip dots/commas/apostrophes, uppercase, collapse whitespace.

2. Build canonical name lookup: `normalized → display_name` from `PLAYERS`.

3. Walk every `SCHEDULE[].matches[].courts[]`:

- Split `home` and `away` strings on `/` to get the two-player pair

- For each player, increment `played`; if their side won, increment `wins`

- Track partner counts: each player's other-side-of-the-slash partner gets `+1`

- Track pair stats keyed by sorted `(p1|p2)` tuple → `{apps, wins}`

4. Assign `gamesPlayed`/`wins` back to `PLAYERS` items by normalized-name lookup

5. Rebuild `PARTNERS` object: for each player, list `[partner_display, count]` sorted desc

6. If `KEY_PAIRS` is provided: for each entry, parse `pair` string, look up in pair-stats, fill in `apps` and `wins`

Name normalization rule

```javascript
norm(s) = s.toUpperCase()
.replace(/[.,''`]/g, ' ') // strip punctuation
.replace(/\s+/g, ' ') // collapse spaces
.trim();
```

This handles cases like `J.SURESH KUMAR` (schedule) ↔ `J. Suresh Kumar` (roster) — both normalize to `J SURESH KUMAR`.

Standings calculation

Each division has its own `renderDiv{X}Standings()` function. The shared logic:

```javascript
const playedPts = s.played; // courts played = max possible pts
const matches = Math.floor(s.played / 3);
const ptsPct = Math.round((s.pts / playedPts) * 100);
```

Exception — Division K: The K data file stores `played` as **matches** (not courts), because the season was scraped late and the source page exposes match counts only. Its render compensates:

```javascript
const matches = s.played;
const playedPts = s.played * 3;
```

Data flow for a weekly refresh

```
league.cdta.co.in
■ (manual scrape via /load-division skill,
■ or admin Data Tools → Auto Fix Stats)
▼
data/division-{a..k}.js ← regenerated, in courts
■
▼
git commit + push
■
▼
```

deploy.bat / deploy.ps1 ← FTP to GoDaddy

■

▼

http://akilanramesh.com/cdta/

```

---

## Admin persistence (server-side)

`api/admin-data.php` is a single PHP endpoint storing JSON files on the GoDaddy server. It is called from the admin panel for:

- User management (`auth.json` mirror)
- Login history append
- Usage log append
- Activity log append
- Settings updates

Auth in PHP is shared-secret based — only the admin role's session token can write.

---

## Browser support

- Modern Chrome / Edge / Safari / Firefox
- Uses CSS Grid, Flexbox, ES2017+ (template literals, arrow functions, optional chaining)
- No polyfills, no IE support

---

## Performance notes

- **Cold load**: ~1.5–3 MB JS payload (mostly the 11 division data files), no minification. Acceptable for an internal tool.
- **Photos**: lazy-loaded by browser; ~64 MB total but each `<img>` only fetches when visible.
- **No virtual DOM**: render functions rebuild `innerHTML` of their target table on every filter change. Tables max ~125 rows so this is fine.
- **Sorting**: in-place array sort, then full re-render.

---
```

Known technical debt

- A single 5,200-line `app.js` is hard to navigate. Future split could be by division.
- Several lint hints for unused variables (`totalPlayed`, `stand`, `teams`, etc.) — pre-existing, harmless.
- Player name normalization is rule-based; rare unicode names may slip through.
- No automated tests — only the manual Test Plan in the admin panel.

Data Schema

Data Schema Reference

Each division's data lives in `data/division-{a..k}.js`. The file declares a set of `const` global variables prefixed with `DIV_X_` (e.g. `DIV_C_PLAYERS`). All globals are loaded into the page via `<script>` tags in `index.html`.

Common structures (all 11 divisions)

`DIV_X_TEAMS`

Array of team objects.

```javascript
const DIV_C_TEAMS = [
{ short:'SVG-C', full:'SVG TENNIS CLUB - C', captain:'Nagendran K', club:'SVG', players:9 },
...
];
```

Field	Type	Notes
`short`	string	Custom short code (see below)
`full`	string	Full team name as on league.cdta.co.in
`captain`	string	Captain's display name
`club`	string	Parent club name

| `players` | number | Roster size |

`DIV_X_PLAYERS`

Array of player objects.

```javascript
{ name:'Nagendran K', team:'SVG-C', gender:'M', age:45, role:'Captain',
gamesPlayed:null, wins:null, photo:'photos/div-c/svg-c/nagendran-k.jpg' }
```

Field	Type	Notes
`name`	string	Display name (Title Case)
`team`	string	Must match a `DIV_X_TEAMS[].short`
`gender`	`'M'` | `'F'` | `null`	
`age`	number | null	
`role`	`'Captain'` | `'Player'`	
`gamesPlayed`	number | null	**Recomputed at runtime** for C/F/H/I
`wins`	number | null	**Recomputed at runtime** for C/F/H/I
`photo`	string	Relative path to JPG/PNG

> **Important:** `gamesPlayed` and `wins` may be `null` in the file. The render layer for divisions C/F/H/I calls `recomputeDivStats()` which derives them from `DIV_X_SCHEDULE`. Do **not** rely on the file values.

`DIV_X_STANDINGS`

Pre-computed standings table (one row per team).

```javascript
{ rank:1, short:'PSG-C', full:'PSG SPORTS CLUB - C', pts:24, played:30,
setsW:48, setsL:18, gamesW:312, gamesL:215 }
```

Field	Type	Notes
`rank`	number	1-based
`short` / `full`	string	
`pts`	number	League points earned
`played`	number	**Courts played** (= max possible pts). Exception: **Div K stores matches**, not courts

| `setsW` / `setsL` | number | Set wins / losses |

| `gamesW` / `gamesL` | number | Game wins / losses |

`DIV_X_SCHEDULE`

Week-by-week match data including court-level player pairings — the **authoritative source of truth** for player stats.

```javascript
const DIV_C_SCHEDULE = [
{ week:1, date:'24-25 Jan 2026', done:true, matches:[
{ home:'PROSCHOOL - B', score:'2-1', away:'CMTA - C', venue:'',
courts:[
{ ct:1, home:'Riyas / Arun Karthik', away:'Manikandan K / NITHISH',
sets:'6-2, 6-0', win:'home' },
...
]},
...
]},
...
];
```

Field	Type	Notes
`week`	number	1-indexed
`date`	string	Free-text date label
`done`	boolean	true if results are in
`matches[]`	array	One per fixture
`matches[].home` / `away`	string	Full team name
`matches[].score`	string	e.g. `'2-1'` (in courts won)
`matches[].courts[]`	array	Always 3 entries (3 doubles per match)
`courts[].ct`	number	Court number (1, 2, or 3)
`courts[].home` / `away`	string	Two player names separated by `/`
`courts[].sets`	string	e.g. `'6-2, 6-0'`
`courts[].win`	`'home'` | `'away'`	Which side won the court

Manual-division-only structures (C, F, H, I)

These four divisions started life as hand-curated data and retain extra fields for projections and key analytics.

`DIV_X_PROJECTIONS`

Projected final standings (synced with actuals once season ends).

```javascript
{ rank:1, short:'PSG-C', full:'PSG SPORTS CLUB - C', currentPts:24, projectedPts:24, change:0 }
```

`DIV_X_PROJ_BREAKDOWN`

Per-team projection breakdown for upcoming weeks. **Empty** when the season is complete.

```javascript
{ team:'PSG-C', remainingMatches:[ { week:11, opponent:'TIR-C', expected:2 }, ... ] }
```

`DIV_X_KEY_PAIRS`

"Most active doubles pairs" — top doubles partnerships by court appearances.

```javascript
{ pair:'Gowtham V / V. Yaswanth', team:'FMP-C', apps:7, wins:null }
```

Field	Type	Notes
`pair`	string	`"Player A / Player B"`
`team`	string	Their team's short code
`apps`	number	Court appearances together
`wins`	number | null	**Recomputed at runtime**

`DIV_X_PARTNERS`

Per-player list of partners and counts. Object keyed by player display name.

```javascript
const DIV_C_PARTNERS = {
```

'Ganesan S': [['Madhan Kumar P',4],['Shravan Krishnan G',1]],

'Madhan Kumar P': [['Ganesan S',4],['Keerthan Kumar B',2]],

...

};

```

> **Note:** This entire object is **rebuilt at runtime** by `recomputeDivStats()`. The static values in the file are placeholders; only the keys (player names) need to be present so the runtime can match them.

### *`DIV_X_TEAM_INFO` (some divisions only)*

Extra metadata per team — court venues, contact, etc. Free-form.

---

## Custom team short codes (manual divisions)

Manual divisions use opinionated short codes to fit in the standings table. The codes are **NOT** scraped — they are hand-assigned and used everywhere as the join key (`DIV_X_PLAYERS.team` → `DIV_X_TEAMS.short`).

| Div C | Div F | Div H | Div I |

|---|---|---|---|

| PSG-C, FMP-C, SNS-A, TIR-C, PRO-C, ANU-C, SVG-C, CMT-C, COS-C, GPT-C, LIF-C | GNEST, ACES, CCTA, ALPHA, SNS, MAP, CSA, CASA, TIR, ALV, CMTA | SNS-D, PSG, ANU-B, RSP, GPTC-C, ANNUR, ACTC-B, TIR-B, KGR-D, FALCON, CSA | SATCH, TTOTS, STARS, RAIL, CBE, TGLO, SNS, GNEST, CHAMPS, SAI, KING |

---

## Photo paths

```

photos/div-{a..k}/{team-short-lower}/{player-name-slug}.{jpg|jpeg|png}

```

- `team-short-lower` is the `short` field lowercased (e.g. `svg-c`)
- `player-name-slug` is the player name lowercased with non-alphanumerics replaced by `-`
- Examples:

- `photos/div-c/svg-c/nagendran-k.jpg`

- `photos/div-i/stars/akilan-ramesh.jpeg`

If a photo is missing the row's `<img onerror>` hides it gracefully.
```

"Courts vs matches" gotcha

Division	What `played` means in `STANDINGS`
A, B, C, D, E, F, G, H, I, J	**Courts played** (matches × 3)
K	**Matches played** (not courts)

The render functions compensate, so the UI always shows `Played Pts (matches)` correctly. If you ever change a render function, copy the pattern from an existing one.

Adding a new player manually

1. Find the team's short code
2. Append to `DIV_X_PLAYERS` with `gamesPlayed:null, wins:null` (the recomputer will fill them)
3. Add the photo file at the correct path
4. Reload — that's it. No other field needs editing.

Adding a new match result manually

1. Find the right `DIV_X_SCHEDULE[wk]` entry
2. Add the match object with all 3 courts and `win` set on each
3. Update `DIV_X_STANDINGS` rows for both teams (rank, pts, played, setsW/L, gamesW/L)
4. Reload — player stats and partners auto-recompute

User Guide

User Guide

This guide is organized by **role**. Find your role and read its section.

Logging in

1. Open http://akilanramesh.com/cdta/
2. Enter your username and password
3. Click **Login**

Your session is remembered until you click **Logout** or clear browser data.

Default accounts

Username	Password	Role	Division	Team
`rameshadmin`	`ramesh123`	admin	all	—
`ramesh`	`ramesh2026`	svgplayer	I	STARS
`monish`	`monish2026`	captain	I	STARS
`murugesan`	`murugesan2026`	captain	I	CHAMPS
`sethupathy`	`sethupathy2026`	captain	F	ACES
`nagendran`	`nagendran2026`	captain	C	SVG-C

Additional users can be created by an admin in **Admin → User Management**.

Common features (all roles except admin extras)

After login you see a top navigation bar with:

```
Overview ■ SVG Players ■ Div A ■ Div B ■ Div C ■ ... ■ Div K ■ [Admin]
```

The **Admin** tab is only visible to the admin role.

Top-level menus

- **Overview** — leaderboard across the whole league

- **Leaders** — top scorers / most active players
- **All Teams** — every team in every division
- **League Stats** — totals row (teams, players, matches, courts)
  - **SVG Players** — special view focused on SVG Tennis Club
- **Club Performance** — default landing
- **Players** / **Schedule** / **Results**
  - **Div A through Div K** — same 5 sub-tabs each:
- **Standings** — final ranks
- **Schedule** — week-by-week fixtures with courts
- **Results** — completed matches
- **Players** — roster with stats
- **Stats** — (C/F/H/I only) — Most Active Doubles Pairs

How to find a player

1. Click any **Div X** in the nav
2. Click **Players** sub-tab
3. Use the search box, team filter, or gender filter
4. The row shows: photo, name, team, courts played, wins/losses, win %, partners

How to read the standings

- **Played Pts** — max possible points (= courts played, since 1 pt per court)
- **Pts** — actual league points earned
- **Win %** — `Pts ÷ Played Pts × 100`
- ■ = champion, ■■ = runners-up, ■ = relegation zone

Tennis terms

- **Match** = team-vs-team fixture (e.g., "PSG-C 2-1 CMTA-C")
- **Court** = one of 3 doubles games inside a match
- **1 court won = 1 point**

Role: viewer

Read-only access to all 11 divisions plus Overview and SVG Players. No write actions, no admin tab.

You can:

- Browse standings, schedules, results, players for any division
- Sort and filter
- View player photos and partner pairings

You cannot:

- Edit any data
- See login history or admin tools

Role: svgplayer

Same as viewer, plus you appear in the **SVG Players** Club Performance view as your own profile (if your name matches a player in the SVG club).

Role: captain

Same as viewer. Captains have no special write permissions in the current build — captain accounts exist so future features (e.g., submitting scores, changing lineup) can be wired up without re-doing auth.

If your team's captain is listed in `data/auth.js`, you can log in with the password assigned by the admin.

Role: admin

Sees everything plus the **Admin** tab. Use the admin tab for all operational tasks.

Admin sub-tabs

Sub-tab	Use it for
User Management	Create, edit, or delete user logins. Set role, division, team.
Login History	Audit who logged in, when, and from where (best-effort)
Usage Log	Page view counts per user
Activity Log	Mutations: data tool runs, refresh actions, etc.
Test Plan	A checklist of manual smoke tests

| **Data Tools** | Auto Fix Stats, Integrity Check, refresh helpers (one button per division) |

| **Settings** | League name, total weeks, photo base path, etc. |

Common admin tasks

- **Add a user**: Admin → User Management → + Add User → fill form → Save.
- **Run a data integrity check**: Admin → Data Tools → "Run Integrity Check" (scans for orphan players, mismatched teams, etc.)
- **Auto-fix stats after a refresh**: Admin → Data Tools → "Auto Fix Stats" (re-derives gamesPlayed/wins from schedule data — same logic that runs at page load)
- **Change league settings**: Admin → Settings → edit values → Save.

For full deployment and refresh procedures see ADMIN_RUNBOOK.md.

Admin Runbook

Admin Runbook

Operational procedures for the CDTA 2026 Dashboard. Read top-to-bottom or jump to a specific task.

Weekly refresh (after match week ends)

For scraped divisions (A, B, D, E, G, J, K)

1. Open Claude Code in the project directory: `cdta\`

2. Run the custom skill: `/load-division X` (replace X with the letter)

3. The skill will:

- Fetch the division page from `https://league.cdta.co.in/`

- Parse standings, players, schedules, results

- Download any new photos to `photos/div-X/`

- Update `data/division-X.js`

4. Reload the dashboard locally and spot-check the new week

5. Commit + push (see Git workflow below)

6. Deploy (see Deployment below)

For manual divisions (C, F, H, I)

These have hand-curated extras. After scraping the base data, you also need to:

- Refresh `DIV_X_PROJECTIONS` (if season ongoing)
- Verify `DIV_X_KEY_PAIRS` still match top pairs
- Photos for new players

The recompute happens automatically — `gamesPlayed`, `wins`, and `PARTNERS` are derived from `DIV_X_SCHEDULE` on every page load. You don't need to update those by hand.

Adding a brand-new division

Use the `/load-division` skill — it has a 10-step automated workflow. Located at `~/.claude/skills/load-division.md`.

After running:

1. Add a new nav button entry in `index.html`
2. Add a new init block in `js/app.js` (copy from another scraped division)
3. Add a `<script src="data/division-X.js">` line in `index.html`

Git workflow

The repo lives on Google Drive, so be aware that Drive sync can briefly lock `.git/objects` during heavy commits. If `git add` fails with "Permission denied", retry — it usually clears in seconds.

```bash
cd "c:/My Cloud/Google Drive/Claude/cdta"
git status
git add <files>
git commit -m "Week N: <division> updates"
git push origin main
```

Only `main` branch is in use. Earlier `workver` branch was deleted on 2026-04-09.

Deployment

See DEPLOYMENT.md for the full FTP / GoDaddy procedure. Short version:

```bat
cd "c:\My Cloud\Google Drive\Claude\cdta"
deploy.bat
```

The script uses credentials from `deploy.env` (not committed). It will FTP all changed files to GoDaddy and report `Deploy complete! N files uploaded.`

Always deploy explicitly when the user says "deploy into godaddy". Never auto-deploy.

Troubleshooting

Player stats showing blank in Players page

Cause: `recomputeDivStats()` did not run, or the player name in `DIV_X_PLAYERS` doesn't match any name in `DIV_X_SCHEDULE` after normalization.

Fix:

1. Open browser console — look for errors at page load

2. Verify the division's init block in `js/app.js` includes `recomputeDivStats(DIV_X_SCHEDULE, DIV_X_PLAYERS, DIV_X_PARTNERS, DIV_X_KEY_PAIRS)`

3. If a specific player is missing, check spelling — normalization strips dots/commas/apostrophes and uppercases. Compare:

- Schedule: `J.SURESH KUMAR` → normalized `J SURESH KUMAR`

- Roster: `J. Suresh Kumar` → normalized `J SURESH KUMAR` ✓

Standings "Played Pts" off by factor of 3

Cause: Render function uses wrong formula. After the courts/matches refactor:

- A–J: `playedPts = s.played` (already in courts)
- K only: `playedPts = s.played * 3` (data is in matches)

Photos not loading

- Check the path matches `photos/div-{x}/{team-short-lower}/{player-slug}.{ext}`
- Verify the file exists in the deployed directory (FTP into GoDaddy and ls)
- The `<img onerror>` hides the element silently — missing photos won't crash the page

Most Active Doubles Pairs shows 0 wins

Cause: `KEY_PAIRS` was not passed to `recomputeDivStats()`, or the pair string doesn't normalize to match the schedule.

Fix:

1. Verify the call includes the 4th argument: `recomputeDivStats(..., DIV_X_KEY_PAIRS)`

2. Check the `pair` string in `DIV_X_KEY_PAIRS` matches the schedule names after normalization

Admin panel data tools fail

Cause: `api/admin-data.php` is unreachable or returned an error.

- Verify the file is deployed to `/cdta/api/admin-data.php` on GoDaddy
- Check browser network tab for the actual error
- Ensure GoDaddy hosting plan supports PHP

Login fails for a known user

- Open `data/auth.js` and verify the username and password
- Auth is checked client-side; case-sensitive

Backups

`data/division-{c,f,h,i}.js.bak` files were saved before the season-end refresh. Keep them in case stats need to be restored.

For everything else, GitHub is the backup. The repo at https://github.com/rameshbala50/cdta is the source of truth.

Credentials and secrets

| Item | Where stored | Notes |

|---|---|---|

| User logins | `data/auth.js` | Plaintext passwords, client-side. Acceptable for an internal league tool. |

| FTP credentials | `deploy.env` (gitignored) | Server: `107.180.115.79`, user: `godaddyftp@akilanramesh.com` |
| GoDaddy hosting | (out of scope) | Manage via GoDaddy account |

Never commit `deploy.env`. It is in `.gitignore`.

Useful file locations

File	Purpose
`index.html`	Single-page DOM
`js/app.js`	All client logic
`css/main.css`	Styles
`data/auth.js`	User credentials
`data/division-{a..k}.js`	Per-division data
`data/division-{c,f,h,i}.js.bak`	Pre-refresh backups
`api/admin-data.php`	Server-side admin storage
`api/fetch-results.php`	Optional CORS proxy for league.cdta.co.in
`deploy.bat` / `deploy.ps1`	GoDaddy FTP deploy
`download-photos.ps1`	Bulk photo downloader from cdta.co.in
`photos/div-{a..k}/`	Local player photos

Deployment

Deployment Guide

The CDTA Dashboard is deployed as static files to **GoDaddy shared hosting** via FTP. There is no build step.

Prerequisites

- Windows machine (deploy scripts are `.bat` and `.ps1`)

- The `deploy.env` file in the project root with FTP credentials (see below)
- Network access to the GoDaddy FTP server

Environment file (`deploy.env`)

This file is **gitignored** and must be created locally. Format:

```
FTP_HOST=107.180.115.79
FTP_USER=godaddyftp@akilanramesh.com
FTP_PASS=<your_password>
FTP_REMOTE_DIR=/cdta
```

A separate `deploy-test.env` exists for the test environment (`/cdta-test` or `/cdta-workver` path).

Deploy to production

Option 1 — Command Prompt

```bat
cd "c:\My Cloud\Google Drive\Claude\cdta"
deploy.bat
```

Option 2 — PowerShell

```powershell
cd "c:\My Cloud\Google Drive\Claude\cdta"
.\deploy.ps1
```

Dry run (PowerShell only)

```powershell
.\deploy.ps1 -DryRun
```

```

---

## What the deploy script does

1. **Cache-bust** — Updates `?v=<timestamp>` query strings in `index.html` so browsers re-fetch JS/CSS
2. **Read credentials** from `deploy.env`
3. **Walk the project tree** — collect every file except the exclusion list
4. **Create directories** on the FTP server (one level at a time, ignoring "already exists" errors)
5. **Upload files** one by one over FTP (passive mode, binary)
6. **Report** OK / FAILED counts

### *Excluded files*

- `deploy.bat`, `deploy.ps1`, `deploy.env`
- `cdta2026_dashboard_w8.html`, `cdta2026_discussion_w8.md` (legacy)
- `.gitignore`
- Anything starting with `.` (e.g., `.git/`)

> **Note:** `.bak` files and `docs/` are **not** in the exclusion list, so they will be uploaded too. Add them to the exclusion list in `deploy.ps1` if you want to keep the production server clean.

---

## Deploy to test environment

```bat
deploy-test.bat
```

This uses `deploy-test.env` and points to a separate folder on the same GoDaddy account so you can preview changes without touching production.

Verifying the deploy

1. Open http://akilanramesh.com/cdta/ in an incognito window (avoids cache)
2. Hard refresh (Ctrl+Shift+R)
```

3. Login as `rameshadmin / ramesh123`

4. Spot-check:

- Overview totals

- Each division's standings

- Players page for one division (photos load, stats populated)

- Admin panel opens

Rollback

If a deploy breaks the site:

1. **Quick rollback** — `git checkout <last_good_commit>` then `deploy.bat`

2. **Or** manually FTP an older snapshot from your local Git history

There is no automated rollback. The `.bak` files in `data/` only cover division data, not full site state.

Photos deployment

Photos live in `photos/div-{a..k}/` and are deployed as part of the normal `deploy.bat` run (1,165 files, ~64 MB). The first deploy after a season refresh takes ~10–20 minutes depending on FTP throughput.

To skip photo upload (much faster), temporarily move the `photos/` folder out of the project before running deploy, then move it back.

Cache busting

`deploy.ps1` rewrites `?v=<timestamp>` in `index.html` on every run. The pattern is:

```html
<script src="js/app.js?v=202604091530"></script>
```

Each deploy bumps the timestamp so browsers fetch the new file. If you ever see stale JS, hard-refresh once — subsequent loads will pick up the new version.

Server-side requirements

GoDaddy shared hosting plan must support:

- **PHP** (for `api/admin-data.php` and `api/fetch-results.php`)
- **FTP write access**
- **Static file serving** (default)

No database, no Node, no Python.

Troubleshooting

"deploy.env not found"

Create the file with your FTP credentials (see format above).

Random "Permission denied" during deploy

GoDaddy occasionally throttles FTP. Re-run the script — it will pick up where it left off (failed files retry).

"FAILED: Connection refused"

Check VPN / firewall. The script uses passive FTP on port 21.

Files uploaded but site shows old content

Hard refresh (Ctrl+Shift+R). The cache-bust query string handles JS/CSS but **not** `index.html` itself, which the browser may still cache aggressively.

Photos missing after deploy

Check the deploy log for "FAILED" entries on `photos/...`. Re-run `deploy.bat` to retry just the failed files (it overwrites existing OK files too — slower but reliable).